

Zynxis Intern Performance Predictor

An End-to-End Machine Learning System for Predicting Intern Success

Final Capstone Project Week 8 report

Prepared by Syeda Mahnoor Fatima

Zynxis Internship Program

1. Executive Summary

This project delivers an end-to-end machine learning system that predicts whether a Zynxis intern is likely to become a high performer and be placed, based on measurable performance signals collected during the internship. The system covers the full pipeline: data preparation, model training and evaluation, and a deployed Streamlit web application that lets a program coordinator enter an intern's metrics and get an instant prediction with a confidence score and a plain-language explanation or screen a whole cohort at once via batch CSV upload.

The final model is a Random Forest Classifier trained on 500 intern records, achieving 69% accuracy and an F1 score of 0.75 on held-out test data. The most influential factors in the prediction are Technical Score and Code Review Score, followed by Soft Skills Rating and Project Completion Rate.

2. Problem Statement

Zynxis runs a structured internship program and tracks a range of performance metrics for each intern technical assessments, project delivery, attendance, soft skills, and code review feedback. Today, deciding which interns are on track to become high performers (and which need additional support) relies on manual review of these metrics by program staff.

The goal of this project is to automate that judgment with a machine learning model: given an intern's metrics at any point in the program, predict the probability that they will end up classified as a high performer / placement-ready candidate. This gives coordinators an early, data driven signal to identify interns who may need extra mentorship, without replacing human judgment.

3. Data Collection & Preparation

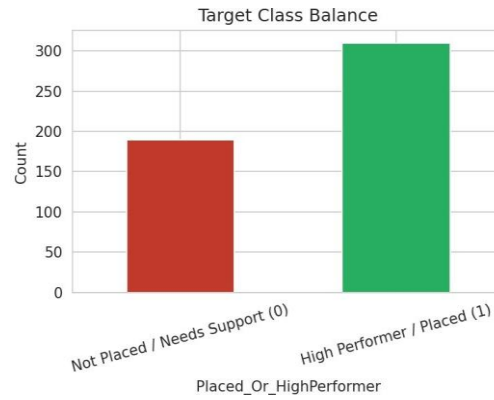
3.1 Data Source

The dataset (zynxis_intern_performance.csv) contains 500 intern records, each with 7 input features and one binary outcome label (Placed_Or_HighPerformer).

- Technical_Score (0–100) = performance on technical coding assessments
- Project_Completion_Rate (%) = share of assigned project work completed
- Attendance_Punctuality (%) = attendance and on-time record
- Soft_Skills_Rating (1–5) = communication and collaboration rating
- Code_Review_Score (1–10) = average code review feedback score
- Prior_Experience_Months (0–24) = relevant experience before the internship
- Education_Level = High_School / Undergrad / Grad

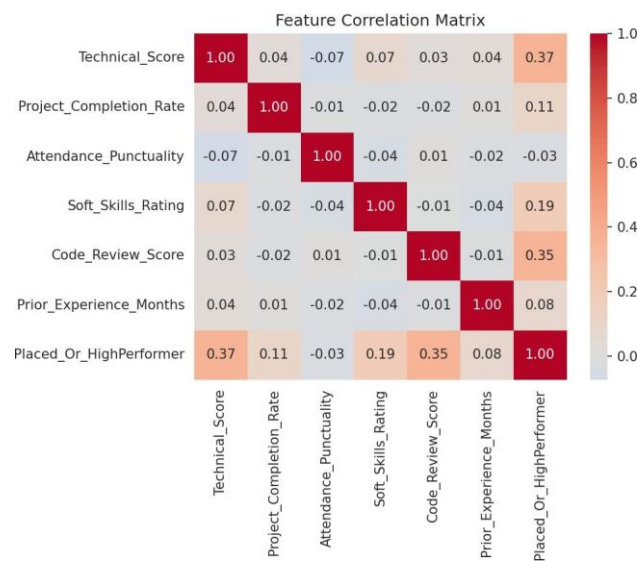
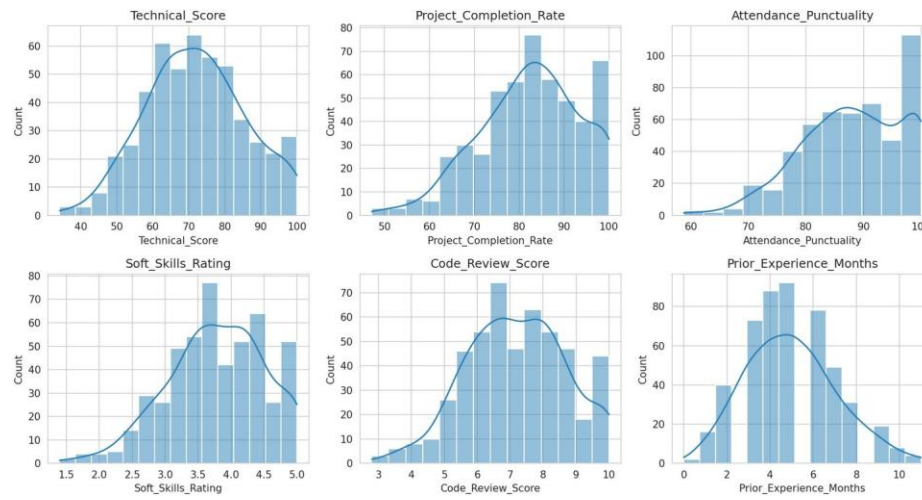
3.2 Data Quality

The dataset was checked for missing values and duplicate records(none were found). The target class is moderately imbalanced: 62% of interns are labeled high performers versus 38% needing support, which was accounted for by using stratified train/test splitting.



3.3 Exploratory Data Analysis

Feature distributions were examined to understand the shape and spread of each metric, and a correlation matrix was used to check for multicollinearity and to see which raw features correlate most with the outcome.



Technical_Score and Code_Review_Score show the strongest positive correlation with the outcome, which foreshadows their importance in the trained model (Section 5.3).

3.4 Preprocessing

- Categorical encoding: Education_Level one-hot encoded (drop_first=True), yielding Education_Level_High_School and Education_Level_Undergrad as binary columns, with Grad as the reference category.
- Feature scaling: all six numeric features standardized with scikit-learn's StandardScaler (fit on the training split only, to avoid data leakage).
- Train/test split: 80/20 stratified split (random_state=42) to preserve class balance in both sets.

4. Model Training

4.1 Algorithm Selection

A Random Forest Classifier was selected as the final model after comparing it against two baselines — Logistic Regression and a single Decision Tree trained on the identical 80/20 split for a fair comparison. Random Forest was chosen for its ability to capture non-linear interactions between features (e.g. technical skill compensating for lower attendance) without heavy manual feature engineering, and for its built-in feature importance scores, which make the model's reasoning interpretable to non-technical stakeholders. All three models are also available side by side in the deployed app's Model Comparison tab (Section 6.3).

Model	Accuracy	F1	ROC-AUC
Random Forest (deployed)	69.0%	0.752	0.710
Logistic Regression	68.0%	0.733	0.715
Decision Tree	65.0%	0.701	0.660

Random Forest and Logistic Regression perform similarly on this dataset, with Random Forest slightly ahead on accuracy and F1; the Decision Tree trails both, consistent with it being more prone to overfitting on a dataset this size. Random Forest was chosen for deployment for its combination of competitive accuracy and built-in feature importance.

4.2 Hyperparameters

- `n_estimators` = 100
- `max_depth` = 7
- `min_samples_split` = 4
- `random_state` = 42 (for reproducibility)

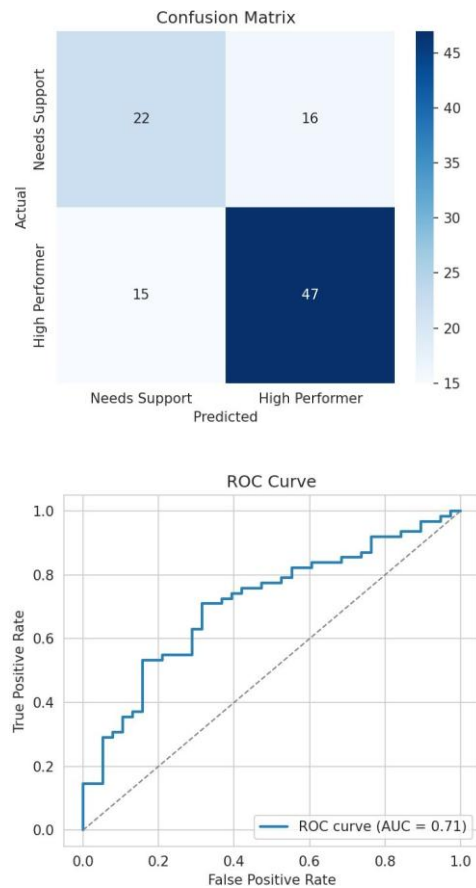
5. Evaluation

5.1 Test Set Metrics

The model was evaluated on the 100 held-out test records (20% split), which it never saw during training.

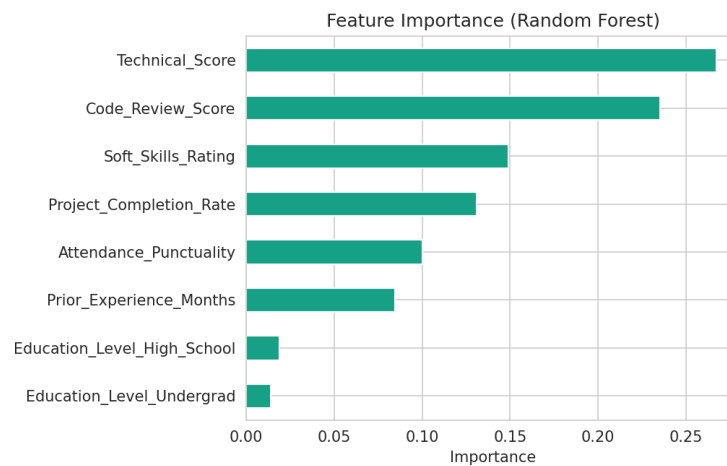
Metric	Score
Accuracy	69.0%
Precision	74.6%
Recall	75.8%
F1 Score	0.752
ROC-AUC	0.710

5.2 Confusion Matrix & ROC Curve



The model performs noticeably better at identifying high performers (recall 76%) than at flagging interns needing support (recall 58%), which is the harder minority-adjacent class given the 62/38 imbalance. An ROC-AUC of 0.71 indicates the model has meaningful, though moderate, discriminative power well above the 0.50 random baseline.

5.3 Feature Importance



Technical_Score and Code_Review_Score dominate the model's decisions, together accounting for roughly half of total feature importance. Education_Level contributes very little the model has learned that demonstrated performance matters far more than formal credentials, which is a reassuring and fair outcome for an internship-evaluation tool.

6. Deployed Interface

The trained model is served through a Streamlit web application (app.py) with four tabs, built to support a program coordinator's actual workflow rather than a single input form.

6.1 Predict

A coordinator enters an intern's six numeric metrics via sliders and selects an education level from a dropdown. The app encodes and scales the input exactly as done at training time, runs it through the saved model, and displays a clear verdict, the predicted probability with a progress bar, a plain language “Why this prediction?” explanation highlighting which top features pushed the result up or down relative to typical values, and an expandable view of the exact feature row sent to the model.

6.2 Batch Prediction

A coordinator can upload a CSV of an entire cohort and receive predictions and probabilities for every intern at once, with a downloadable results file and a summary count of how many interns fall into each outcome class — turning a one-at-a-time tool into something usable for real program-level screening.

6.3 Model Comparison

Logistic Regression and a Decision Tree were trained on the identical data split as the deployed Random Forest, and all three are shown side by side in the app (accuracy, precision, recall, F1, ROC-AUC). This makes the model-selection decision from Section 4.1 verifiable inside the product itself, not just asserted in this report.

6.4 Feature Importance

An interactive chart of the deployed model's feature weights, with the same plain-language takeaway as Section 5.3, so a non-technical stakeholder can see what drives the tool's recommendations without reading this document.

7. Limitations & Future Work

- Dataset size (500 records) is modest; a larger, real-world sample would likely improve generalization and reduce variance in the minority-class recall.
- 69% accuracy means roughly 3 in 10 predictions are wrong the tool should support, not replace, a coordinator's judgment.
- Recall on the “Needs Support” class (58%) is the weakest metric; techniques like class-weighting or SMOTE oversampling could be explored to catch more at-risk interns.
- The “Why this prediction?” explanation compares an intern's values to typical dataset ranges; a future version could use SHAP values for a mathematically precise, per-prediction breakdown of each feature's exact contribution.
- Batch prediction currently expects a clean CSV with exact column names; a future version could add fuzzy column matching and inline validation feedback for messier real-world exports.

8. Conclusion

This project demonstrates a complete, working machine learning pipeline for Zynxis from raw intern performance data through a trained, evaluated Random Forest model to a deployed, user-friendly prediction tool. The model surfaces technical performance and code quality as the dominant predictors of intern success, giving Zynxis a concrete, data-backed signal to complement its existing internship evaluation process.